

ALGORITHMS FOR HYBRID MODELS AND THEIR EXPLAINABILITY

Table of Contents

ALGORITHMS FOR HYBRID MODELS AND THEIR EXPLAINABILITY 1

Algorithm 1A – Hybrid A (PFD-GSTE Variant A) Forward Pass 2

Algorithm 1B – Hybrid B (PFD-GSTE Variant B) Forward Pass 3

Algorithm XAI-2A – XAI for Hybrid A (PFD-GSTE Variant A) 5

Algorithm XAI-2B – XAI for Hybrid B (PFD-GSTE Variant B)..... 7

Algorithm 1A – Hybrid A (PFD-GSTE Variant A) Forward Pass

Input: image batch x , flag return_xai

Output: logits, and optionally {"mask": mask_img, "attn": attn_list}

1. $\text{feat} \leftarrow \text{CNN}(x)$ (ResNet50V2 feature map, e.g., (B,2048,7,7))
2. $\text{z_cnn} \leftarrow \text{AvgPool}(\text{feat})$
3. $\text{z_cnn} \leftarrow \text{Flatten}(\text{z_cnn})$
4. $\text{z_cnn} \leftarrow \text{ReLU}(\text{Linear_cnn_proj}(\text{z_cnn}))$
5. $(\text{feat_path}, \text{mask_feat}) \leftarrow \text{PFD}(\text{feat})$
 - $\text{mask_feat} \leftarrow \text{Sigmoid}(\text{Conv1x1}(\text{feat}))$
 - $\text{feat_path} \leftarrow \text{feat} * \text{mask_feat}$
6. $\text{mask_img} \leftarrow \text{BilinearUpsample}(\text{mask_feat}, \text{size} = \text{input_image_size})$ (only for visualisation/XAI)
7. $\text{token_sets} \leftarrow \text{empty list}$
8. **For each** (rot_idx, k) **in** enumerate(rotations) **do**
 - 8.1 $\text{f_r} \leftarrow \text{rot90}(\text{feat_path}, k)$
 - 8.2 $\text{m_r} \leftarrow \text{rot90}(\text{mask_feat}, k)$
 - 8.3 $(\text{tokens}, \text{ht}, \text{wt}) \leftarrow \text{FeatureTokenEmbed}(\text{f_r})$
 - 8.4 $\alpha \leftarrow \text{Flatten}(\text{m_r})$ as (B, ht*wt, 1)
 - 8.5 $\alpha \leftarrow \alpha / (\text{mean}(\alpha \text{ over tokens}) + 1e-6)$
 - 8.6 $\text{tokens} \leftarrow \text{tokens} * \alpha$ (GSTE-A: static guidance weighting)
 - 8.7 $\text{tokens} \leftarrow \text{PosRotEmbed}(\text{tokens}, \text{ht}, \text{wt}, \text{rot_idx})$
 - 8.8 append tokens to token_sets**Endfor**
9. $\text{Tavg} \leftarrow \text{Mean}(\text{token_sets} \text{ over rotations})$
10. $(\text{Tenc}, \text{attn_list}) \leftarrow \text{RViTEncoder}(\text{Tavg}, \text{return_attn} = \text{return_xai})$
11. $\text{z_vit} \leftarrow \text{Mean}(\text{Tenc} \text{ over tokens})$
12. $\text{z_vit} \leftarrow \text{ReLU}(\text{Linear_vit_proj}(\text{z_vit}))$
13. $\text{z} \leftarrow \text{Concat}(\text{z_cnn}, \text{z_vit})$
14. $\text{h} \leftarrow \text{ReLU}(\text{Linear_fuse_fc}(\text{z}))$
15. $\text{h} \leftarrow \text{Dropout}(\text{h})$
16. $\text{logits} \leftarrow \text{Linear_out}(\text{h})$
17. **If** return_xai **then**
 - Return** logits, {"mask": mask_img, "attn": attn_list}
 - Else**
 - Return** logits, None**Endif**

Algorithm 1B – Hybrid B (PFD-GSTE Variant B) Forward Pass

Input: image batch x , flag return_xai

Output: logits, and optionally {"attn": attn_list, "mask": mask_img, "gste_side": dyn_side}

1. $\text{feat} \leftarrow \text{CNN}(x)$ (ResNet50V2 feature map, e.g., (B,C,7,7))
2. **If** $\text{use_pfd_gste} == \text{True}$ **then**
 - 2.1 $(\text{feat_path}, \text{mask_feat}) \leftarrow \text{PFD}(\text{feat})$
 - 2.2 $\text{feat_for_cnn} \leftarrow \text{feat_path}$
 - 2.3 $\text{mask_img} \leftarrow \text{BilinearUpsample}(\text{mask_feat}, \text{size} = \text{input_image_size})$**Else**
 - 2.4 $\text{feat_for_cnn} \leftarrow \text{feat}$
 - 2.5 $\text{mask_img} \leftarrow \text{None}$**Endif**
3. $\text{cache_last_cnn_feat} \leftarrow \text{feat_for_cnn}$
4. $\text{cache_last_cnn_mask_img} \leftarrow \text{mask_img}$
5. **CNN descriptor head**
 - 5.1 $\text{z_cnn} \leftarrow \text{AvgPool}(\text{feat_for_cnn})$
 - 5.2 $\text{z_cnn} \leftarrow \text{Flatten}(\text{z_cnn})$
 - 5.3 $\text{z_cnn} \leftarrow \text{Dropout}(\text{z_cnn})$
 - 5.4 $\text{z_cnn} \leftarrow \text{Linear_cnn_proj}(\text{z_cnn})$
 - 5.5 $\text{z_cnn} \leftarrow \text{BatchNorm}(\text{z_cnn})$
 - 5.6 $\text{z_cnn} \leftarrow \text{ReLU}(\text{z_cnn})$
6. **Choose dynamic token grid side (GSTE decision)**
If $\text{use_pfd_gste} == \text{True}$ **then**
 - 6.1 $\alpha_0 \leftarrow \text{AdaptiveAvgPool}(\text{mask_img}, \text{output}=(g,g))$ ($g = \text{base_grid} = \text{img_size} // \text{patch_size}$)
 - 6.2 $\alpha_0 \leftarrow \alpha_0 / (\text{mean}(\alpha_0 \text{ over grid}) + 1e-6)$
 - 6.3 $\text{dyn_side} \leftarrow \text{choose_dynamic_side}(\alpha_0)$ where:
 - $s \leftarrow \text{mean}(\text{std}(\alpha_0 \text{ over grid cells}))$
 - **If** $s \leq \text{gste_std_flat}$ **then** $\text{dyn_side} = g$
 - **Else**
 - $t \leftarrow \text{clamp}((s - \text{gste_std_flat}) / (\text{gste_std_full} - \text{gste_std_flat}), 0, 1)$
 - $\text{shrink} \leftarrow \text{gste_max_shrink} * t$
 - $\text{target} \leftarrow \text{round}(g * (1 - \text{shrink}))$
 - $\text{dyn_side} \leftarrow \text{clamp}(\text{target}, \text{gste_min_side}, g)$
 - 6.4 $\text{dyn_side} \leftarrow g$**Endif**
7. $\text{token_sets} \leftarrow \text{empty list}$
8. **For each** k **in** rotations **do**
 - 8.1 $\text{rot_id} \leftarrow k \bmod 4$
 - 8.2 $x_r \leftarrow \text{rot90}(x, \text{rot_id})$
 - 8.3 $(\text{tokens_base}, \text{ht}, \text{wt}) \leftarrow \text{ImagePatchEmbed}(x_r)$ (tokens on $g \times g$ grid)
 - 8.4 **If** $\text{use_pfd_gste} == \text{True}$ **then**
 - 8.4.1 $m_r \leftarrow \text{rot90}(\text{mask_img}, \text{rot_id})$
 - 8.4.2 $\alpha_{\text{base}} \leftarrow \text{AdaptiveAvgPool}(m_r, \text{output}=(g,g))$
 - 8.4.3 $\alpha_{\text{base}} \leftarrow \alpha_{\text{base}} / (\text{mean}(\alpha_{\text{base}} \text{ over grid}) + 1e-6)$
 - 8.4.4 $(\text{tokens}, \text{ht}, \text{wt}) \leftarrow \text{GSTE_dynamic_tokens}(\text{tokens_base}, \alpha_{\text{base}}, \text{dyn_side})$ where:
 - $\text{reshape } \text{tokens_base} \rightarrow (B, D, g, g)$
 - multiply by α_{base} (ROI weighting)
 - **If** $\text{dyn_side} < g$ **then** weighted pool to $(\text{dyn_side}, \text{dyn_side})$ using (num/den)
 - flatten back $\rightarrow (B, \text{dyn_side} * \text{dyn_side}, D)$

```
8.5 tokens  $\leftarrow$  tokens_base  
Endif
```

```
8.6 tokens  $\leftarrow$  PosRotEmbed(tokens, ht, wt, rot_id)  
8.7 append tokens to token_sets  
Endfor
```

```
9. Tavg  $\leftarrow$  Mean(token_sets over rotations)  
10. (Tenc, attn_list)  $\leftarrow$  RViTEncoder(Tavg, return_attn = return_xai)  
11. z_vit  $\leftarrow$  Mean(Tenc over tokens)  
12. z_vit  $\leftarrow$  Linear_vit_proj(z_vit)  
13. z_vit  $\leftarrow$  ReLU(z_vit)  
14. z  $\leftarrow$  Concat(z_cnn, z_vit)  
15. h  $\leftarrow$  Linear_fuse_fc(z)  
16. h  $\leftarrow$  ReLU(h)  
17. h  $\leftarrow$  Dropout(h)  
18. logits  $\leftarrow$  Linear_out(h)  
19. If return_xai then  
    Return logits, {"attn": attn_list, "mask": mask_img, "gste_side": dyn_side}  
    Else  
    Return logits, None  
    Endif
```

Algorithm XAI-2A – XAI for Hybrid A (PFD-GSTE Variant A)

Input: --checkpoint, --image, --out_dir, --mc_samples, --target_class

Output: saved overlays (xai_gradcamp.png, xai_attn_rollout.png) and printed prediction summary

1. Parse CLI arguments: checkpoint, image, out_dir, mc_samples, target_class
2. device $\leftarrow$ "cuda" if available else "cpu"
3. Create out_dir if it does not exist
4. Load checkpoint ckpt $\leftarrow$ torch.load(checkpoint, map_location=device)
5. Read from checkpoint (with fallbacks):
 - o class_names $\leftarrow$ ckpt["class_names"] else ["glioma", "meningioma", "pituitary", "notumor"]
 - o mean $\leftarrow$ ckpt["mean"] else [0.485, 0.456, 0.406]
 - o std $\leftarrow$ ckpt["std"] else [0.229, 0.224, 0.225]
 - o cfg $\leftarrow$ ckpt["model_cfg"] else {}
6. Build Hybrid A model using cfg (with defaults) and set cnn_pretrained=False
7. Load weights:
 - o state $\leftarrow$ ckpt["model_state"] if exists else ckpt["state_dict"]
 - o If state missing $\rightarrow$ raise error
 - o model.load_state_dict(state)
8. model.eval()
9. Load image using PIL, convert to RGB, resize to (224,224)
10. Preprocess image:
 - o Convert to tensor in [0,1], permute to (3,H,W)
 - o Normalize using mean,std
 - o Add batch dimension $\rightarrow$ x shaped (1,3,224,224)
 - o Move x to device
11. Prepare a dictionary hook_cache $\leftarrow$ {}
12. Define **PFD forward hook** _pfd_hook(module, inputs, output):
 - o feat_path $\leftarrow$ output[0] (*PFD returns (feat_path, mask_feat)*)
 - o feat_path.retain_grad()
 - o hook_cache["cnn_feat"] $\leftarrow$ feat_path
13. Register hook: hook_handle $\leftarrow$ model.pfd.register_forward_hook(_pfd_hook)
14. Run forward with gradients enabled:
 - o with torch.enable_grad():
 - (logits, xai) $\leftarrow$ model(x, return_xai=True)
15. Remove hook: hook_handle.remove()
16. Compute prediction:
 - o prob $\leftarrow$ softmax(logits)[0]
 - o pred_idx $\leftarrow$ argmax(prob)
 - o conf $\leftarrow$ prob[pred_idx]
17. Choose explanation class:
 - o If target_class < 0 then target $\leftarrow$ pred_idx
 - o Else target $\leftarrow$ target_class
18. Backprop for Grad-CAM+:
 - o model.zero_grad(set_to_none=True)
 - o logits[0, target].backward(retain_graph=True)
 - o A $\leftarrow$ hook_cache["cnn_feat"]
 - o If A missing $\rightarrow$ raise error
19. Build **Grad-CAM++** heatmap:
 - o cam_small $\leftarrow$ gradcam_pp_from_activations(A, A.grad) (*feature-map resolution*)
 - o Upsample to (224,224) using bilinear interpolate $\rightarrow$ cam
 - o Convert cam to NumPy for plotting
20. Build **Attention Rollout** heatmap:
 - o attn_list $\leftarrow$ xai["attn"] if exists else xai["attn_list"]
 - o If missing $\rightarrow$ raise error
 - o attn_tok $\leftarrow$ attention_rollout(attn_list) (*returns ht $\times$ wt token map inferred from N*)
 - o Upsample attn_tok to (224,224) using bilinear interpolate $\rightarrow$ attn_img
 - o Convert attn_img to NumPy

21. MC Dropout uncertainty:

- with torch.no_grad(): (mu, var) ← model.mc_dropout_predict(x, mc_samples)
- Convert to NumPy
- mu_pred ← argmax(mu)
- mu_conf ← mu[mu_pred]
- mu_var ← var[mu_pred]

22. Save overlays:

- overlay_and_save(img_rgb, cam, out_dir/"xai_gradcampp.png", "Grad-CAM++ (target=...)")
- overlay_and_save(img_rgb, attn_img, out_dir/"xai_attn_rollout.png", "Attention Rollout (token relevance)")

23. Print:

- single-pass predicted class, confidence
- MC-dropout mean class, mean confidence, predictive variance
- saved file paths

Algorithm XAI-2B – XAI for Hybrid B (PFD-GSTE Variant B)

Input: --checkpoint, --image, --out_dir, --mc_samples, --target_class

Output: saved overlays (xai_gradcamp.png, xai_attn_rollout.png) and printed prediction summary

1. Parse CLI arguments: checkpoint, image, out_dir, mc_samples, target_class
2. Force matplotlib headless backend: `matplotlib.use("Agg")`
3. `device ← "cuda" if available else "cpu"`
4. Create out_dir if it does not exist
5. Load checkpoint `ckpt ← torch.load(checkpoint, map_location=device)`
6. Read from checkpoint (with fallbacks):
 - `class_names ← ckpt["class_names"]` else `["glioma", "meningioma", "pituitary", "notumor"]`
 - `mean ← ckpt["mean"]` else `[0.5, 0.5, 0.5]`
 - `std ← ckpt["std"]` else `[0.5, 0.5, 0.5]`
 - `cfg ← ckpt["model_cfg"]` else `{}`
7. Build Hybrid B model from cfg and set:
 - `cnn_pretrained=False`
 - `use_pfd_gste=True` (*critical: ensures mask + dynamic tokens exist*)
8. Load weights:
 - `state ← ckpt["model_state"]` if exists else `ckpt["state_dict"]`
 - If missing → raise error
 - `model.load_state_dict(state)`
9. `model.eval()`
10. Load image using PIL, convert to RGB, resize (224,224)
11. Preprocess image exactly like script:
 - Tensor in [0,1], permute to (3,H,W)
 - Normalize by mean,std
 - Add batch dim → (1,3,224,224)
 - Move to device
12. Prepare `hook_cache ← {}`
13. Define **PFD forward hook** `_pfd_hook(module, inputs, output)`:
 - Verify output is tuple/list
 - `feat_path ← output[0]`
 - `feat_path.retain_grad()`
 - `hook_cache["cnn_feat"] ← feat_path`
14. Register hook: `h ← model.pfd.register_forward_hook(_pfd_hook)`
15. **Try / Finally** (so hook is always removed):
 - **Try**
 - 15.1 with `torch.enable_grad()`: `(logits, xai) ← model(x, return_xai=True)`
 - 15.2 If xai not dict → raise error
 - 15.3 If `xai["mask"]` is None → raise error (*script enforces PFD/GSTE active*)
 - 15.4 `prob ← softmax(logits)[0]`
 - 15.5 `pred_idx ← argmax(prob)` and `conf ← prob[pred_idx]`
 - 15.6 If `target_class < 0` then `target ← pred_idx` else `target ← target_class`
 - 15.7 `model.zero_grad(set_to_none=True)`
 - 15.8 `logits[0, target].backward()`
 - 15.9 `A ← hook_cache["cnn_feat"]`
 - 15.10 If A missing or A.grad missing → raise error
 - **Finally**
 - 15.11 `h.remove()`
16. Build **Grad-CAM++** heatmap:
 - `cam_small ← gradcam_pp_from_activations(A, A.grad)`
 - Upsample to (224,224) using bilinear interpolate → `cam`
 - Convert cam to NumPy
17. Build **Attention Rollout** heatmap (uses GSTE side if available):
 - `attn_list ← xai["attn"]` if exists else `xai["attn_list"]`
 - If missing → raise error
 - `gste_side ← xai.get("gste_side", None)`

- `attn_tok ← attention_rollout(attn_list, side=gste_side)`
 - If side is provided, script **requires** `side*side == N` or raises error
 - Upsample `attn_tok` to (224,224) → `attn_img`
 - Convert `attn_img` to NumPy
18. MC Dropout uncertainty:
- with `torch.no_grad()`: `(mu, var) ← model.mc_dropout_predict(x, mc_samples)`
 - Convert to NumPy
 - `mu_pred ← argmax(mu)`
 - `mu_conf ← mu[mu_pred]`
 - `mu_var ← var[mu_pred]`
19. Save overlays:
- `overlay_and_save(img_rgb, cam, out_dir/"xai_gradcampp.png", "Grad-CAM++ (PFD target=...)"`)
 - `overlay_and_save(img_rgb, attn_img, out_dir/"xai_attn_rollout.png", "Attention Rollout")`
20. Print:
- single-pass predicted class, confidence
 - MC-dropout mean class, mean confidence, predictive variance
 - saved file paths